Linear Regression by Gradient Descent

Predicting red wine quality from physicochemical data

MAI391 — Mathematics for Machine Learning · Computer Project · Group 1

Nguyễn Hoàng Phúc — SE203130

Nguyễn Minh Bảo — SE203940

Trương Huy An — SE210173

1. The problem and the data

- **Question** (Cortez et al., 2009): can wine sensory scores be predicted from lab measurements, replacing costly, subjective tasting?
- **Data:** UCI Wine Quality, red *vinho verde* — 1599 samples, 11 numeric features (acidity, sugar, chlorides, SO_2, density, pH, sulphates, alcohol).
- **Target:** *quality*, integer score (only 3–8 appear); strongly concentrated on 5 and 6 ($> 80\%$).
- **Split:** random, seed 42 → 1279 train / 320 test. No missing values, CC BY 4.0.
- **Approach:** linear regression solved by **gradient descent written from scratch in NumPy** — chosen because the closed form gives a ground truth to test convergence theory, and feature scales differ hugely (total SO_2 std ≈ 33, density ≈ 0.002), a classic conditioning example.

- Wine experts score quality from 0 to 10, which is slow, costly and subjective.
- We ask: can 11 numbers measured in a lab (alcohol, acidity, sulphates…) predict that score? We use the simplest model, a weighted sum of the features, and find the best weights with gradient descent, an algorithm that improves them step by step.

2. Model: from Gaussian likelihood to least squares

- Add intercept: $\tilde{x}_n = (1, x_n)$, $D = 11$. Assume $y_n = \tilde{x}_n^T\theta + \varepsilon_n$, $\varepsilon_n \sim N(0, \sigma^2)$ i.i.d.
- Negative log-likelihood: $-\log p(y|X,\theta) = (1/2\sigma^2)\|y - X\theta\|^2 + \text{const.}$
- Maximum likelihood $\Leftrightarrow$ minimise squared error (σ^2 only rescales).

- We predict the score as a weighted sum of the features plus a constant (θ holds the
- weights).
- If the errors are random bell-curve noise, the best weights are those with the smallest total squared error. So $L(\theta)$ is just the average squared prediction error (halved): the "cost" we want to minimise.

$$\mathbf{L(\theta) = (1/2N)\,\|y - X\theta\|^2},\ \theta \in \mathbb{R}^{D+1}$$

Shapes: X is $N\times(D+1) = 1279\times12$; y is N; θ, ∇L are $D+1$; γ, β, $L(\theta)$ are scalars.

3. Gradient, Hessian and convexity

- Expanding $L(\theta)$ gives:

$$\nabla L(\theta) = (1/N)\, X^T(X\theta - y)$$

$$H = \nabla^2 L = (1/N)\, X^T X$$

- **Proposition:** $v^T H v = (1/N)\|Xv\|^2 \geq 0 \Rightarrow L$ is convex; if rank $X = D+1$ then $H \succ 0$, so L is strictly convex with a **unique global minimiser**.
- **Normal equations:** $X^T X\theta^* = X^T y \Rightarrow \theta^* = (X^T X)^{-1} X^T y$ — $X\theta^*$ is the orthogonal projection of y onto the column space of X.
- Exact quadratic: $L(\theta) = L(\theta^*) + \frac{1}{2}(\theta-\theta^*)^T H(\theta-\theta^*)$.
- The gradient points in the direction where the error grows fastest, so we move the
- opposite way. The Hessian H describes the curvature of the error surface.
- Since it is never negative, the surface is a single bowl-shaped valley: exactly one lowest point, and that point is the best set of weights.

4. Gradient descent and the convergence theorem

- Update: $\boldsymbol{\theta_{t+1} = \theta_t - \gamma \nabla L(\theta_t)}$. Let $\mu = \lambda_{min}(H)$, $\ell = \lambda_{max}(H)$, $\kappa = \ell/\mu$.
- **Theorem:** GD converges for every θ_0 **iff** $\boldsymbol{0 < \gamma < 2/\ell}$, with $\|\theta_t - \theta^*\| \le \rho^t \|\theta_0 - \theta^*\|$, $\rho = \max\{|1-\gamma\mu|, |1-\gamma\ell|\}$.
- **Proof idea:** error $e_{t+1} = (I - \gamma H)e_t$; diagonalise $H = Q\Lambda Q^T$ (spectral theorem) → each eigen-direction is multiplied by $(1 - \gamma\lambda_i)$.
- Optimal step $\gamma_{opt} = 2/(\ell+\mu)$ gives $\rho = (\kappa-1)/(\kappa+1)$; $\gamma = 1/\ell$ gives $\rho = 1 - 1/\kappa$.
- **Takeaway:** the step size is capped by ℓ; the speed is decided by κ (iterations $\approx \kappa \cdot \ln(\|e_0\|/\varepsilon)$).

- Gradient descent is like walking downhill in fog: at each step move a little in the steepest downward direction; γ is the step size.
- Too big a step overshoots and blows up ($\gamma \ge 2/\ell$); too small is very slow. μ and ℓ are the smallest and largest curvature; $\kappa = \ell/\mu$ measures how stretched the valley is.

  Long, narrow valleys (large κ) need many steps.

5. Standardisation and conditioning

- Standardise $z = (x - m)/s$ with **training-set** statistics only (no leakage). Then $H = \mathrm{diag}(1, C)$, C = correlation matrix.
- $\mathrm{tr}\, H = D+1 \Rightarrow$ eigenvalues lie in $[0, D+1]$; κ now reflects only **multicollinearity**.
- Raw data: large means inflate λ_{max}, tiny-variance directions (density) make λ_{min} tiny $\rightarrow \kappa$ huge, γ forced small.
- Same affine model class $\Rightarrow$ same optimal loss and predictions; only the optimisation path changes.

Features have very different units (total SO_2 is in tens, density in thousandths).

- Rescaling every feature to mean 0 and spread 1 turns a long narrow valley into a round bowl.
- The same algorithm goes from practically never finishing ($\kappa \approx 10^{10}$) to finishing in about 500 steps ($\kappa \approx 52$).
- Test data is rescaled with the training mean/std to avoid leaking information.

Raw data: $\kappa \approx 1.29 \times 10^{10}$

Standardised: $\kappa \approx 51.6$

6. Heavy-ball momentum

- $v_{t+1} = \beta v_t - \gamma \nabla L(\theta_t)$, $\theta_{t+1} = \theta_t + v_{t+1}$ ($\beta = 0$ is plain GD).
- Per eigen-direction: $e_{t+1} = (1+\beta-\gamma\lambda)e_t - \beta e_{t-1}$; characteristic roots are complex with modulus $\sqrt{\beta}$ when $(1-\sqrt{\beta})^2 \le \gamma\lambda \le (1+\sqrt{\beta})^2$.
- Matching this interval to $[\gamma\mu, \gamma\ell]$ gives the optimal (Polyak) parameters:
- Momentum works like a rolling ball: it keeps part of its previous velocity (β), so it speeds up along
- the valley instead of zig-zagging across it.

  With well-chosen γ and β, the number of steps needed drops from about κ to about $\sqrt{\kappa}$, a huge saving when κ is large.

$$\gamma = 4 / (\sqrt{\ell} + \sqrt{\mu})^2,\ \beta = ((\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1))^2,\ \text{rate } \sqrt{\beta} \approx 1 - 2/\sqrt{\kappa}$$

Momentum replaces κ by $\sqrt{\kappa}$ in the iteration count. For $\kappa = 51.6$: $\sqrt{\beta} \approx 0.756$ versus $\rho_{opt} \approx 0.962$ for plain GD.

7. Implementation

- **Core** gd_linreg.py: NumPy only, fully vectorised, float64; two matrix–vector products per iteration.
- **Algorithm:** $\theta \leftarrow 0$, $v \leftarrow 0$; repeat: $g \leftarrow (1/N)X^T(X\theta - y)$; stop if $\|g\| < \varepsilon$; $v \leftarrow \beta v - \gamma g$; $\theta \leftarrow \theta + v$; stop if loss non-finite or $> 10^{30}$ (divergence guard).
- λ_{max}, λ_{min} from eigvalsh(H) to set γ; closed form via solve (not explicit inverse).
- **Reproducible:** split with default_rng(42), $\theta_0 = 0$. scikit-learn used only for cross-checking.
- **Cost:** O(TND) for GD vs $O(ND^2 + D^3)$ for normal equations — GD pays off for very large D or streaming data.

- Everything is written from scratch with NumPy matrix operations.
- Each loop computes the gradient, updates the weights, and stops when the gradient is almost zero or when the loss explodes (divergence).
- A fixed random seed (42) means anyone can re-run the code and get identical numbers.

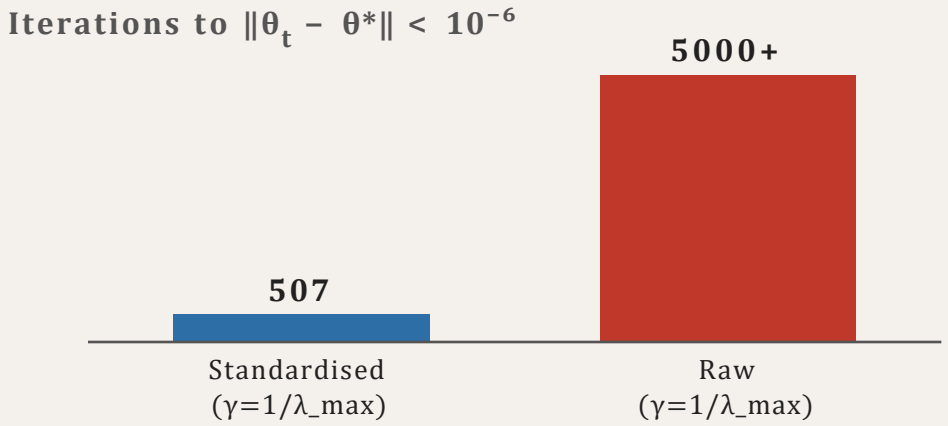
8. Verification, step by step

- **1. Gradient check:** analytic ∇L vs central finite differences at a random $\theta \rightarrow$ max difference **3.6** $\times$ $\mathbf{10^{-10}}$.
- **2. Synthetic data:** $y = X\theta_{true}$ + noise ($\sigma = 0.05$, $N = 500$). GD matches normal equations to **6.7** $\times$ $\mathbf{10^{-16}}$; distance to θ_{true} is 0.0078, the order of the statistical error $\sigma/\sqrt{N} \approx 0.002$ — not an algorithm error.
- **3. Real data:** only after the first two tests passed.

- We tested in layers so mistakes cannot hide: (1) compare our gradient formula with a numerical estimate; (2) test on fake data where the true weights are known; (3) only then run the real wine data.
- Each check passed, so later results can be trusted.

9. Raw vs standardised data

Data	λ_max	λ_min	κ	$\gamma_max = 2/\lambda_max$
Raw	3.63×10^{3}	2.82×10^{-7}	1.29×10^{10}	5.51×10^{-4}
Standardised	3.084	0.0597	51.6	0.6485



- Standardisation cuts κ by $\approx 2.5 \times 10^{8}$.
- Standardised: converged in **507** iterations.
- Raw: after 5000 iterations loss gap still 0.030, $\|\theta-\theta^*\| \approx 57.4$, test MSE 0.5001 vs 0.4018 optimal; theory needs $\sim10^{11}$.

- Same algorithm, same step rule; only the feature scaling differs.
- After standardising it reaches the exact answer in 507 steps; on raw data it is still far away after 5000 steps (test error 0.50 instead of the optimal 0.40).
- Lesson: always standardise features before gradient descent.

10. Step-size study (standardised data)

γ	γ/γ_{max}	$L(\theta_{300})$	Iters to 10^{-6}
$0.1/\lambda_{max}$	0.05	0.211270	5114
$0.5/\lambda_{max}$	0.25	0.211221	1019
$1.0/\lambda_{max}$	0.50	0.211220	507
$1.9/\lambda_{max}$	0.95	0.211220	265
$1.99/\lambda_{max}$	0.995	0.211233	1090
$2.05/\lambda_{max}$	1.025	diverges	—

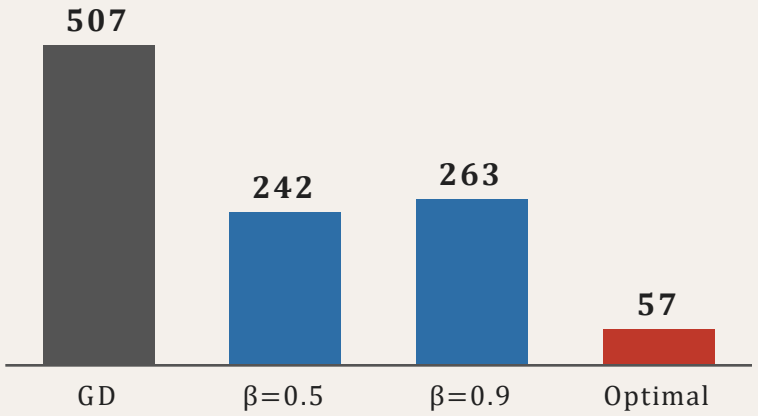
- $\gamma > 2/\lambda_{max}$ **diverges** (matches theorem).
- γ close to $2/\lambda_{max}$: $\rho = |1-\gamma\lambda_{max}| \approx 0.99 \rightarrow$ slow.
- Tiny γ: $\rho = 1-\gamma\mu \approx 1 \rightarrow$ slow.
- Theory: $\gamma_{opt} = 0.6362$, $\rho = 0.962 \rightarrow \leq 401$ iterations (measured 285). For $\gamma = 1/\lambda_{max}$ bound ≈ 803 (measured 507): bounds hold.

$L(\theta^*) = 0.2112205$

- γ must stay below $2/\lambda_{max}$.
- At $2.05/\lambda_{max}$ the loss explodes (diverges); close to $2/\lambda_{max}$, or with a very small γ, progress is
- slow. The best values lie in between.
- "Iters" = steps needed to get within 10^{-6} of the exact
- solution. Measured steps always stay below the theoretical bounds.

11. Effect of momentum

Configuration	γ	β	Iters to 10^{-6}
Plain GD (β=0)	0.3243	0.000	507
Momentum β=0.5	0.3243	0.500	242
Momentum β=0.9	0.3243	0.900	263
Heavy-ball optimal	0.9995	0.571	57



- Optimal heavy-ball ≈ **9× faster** than GD; theory ≈ 56, measured 57 — $\kappa \to \sqrt{\kappa}$ confirmed.
- β = 0.9 oscillates (complex roots), as predicted.

- A little momentum (β = 0.5) already halves the number of steps.
- The theory-optimal setting needs only 57 steps instead of 507, about 9× faster.
- Too much momentum (β = 0.9) overshoots and oscillates, so the parameters must be chosen carefully.

12. Comparison with a standard library

Method	Train MSE	Test MSE	Test R^2	$\|\theta-\theta^*\|_\infty$
Normal equations (NumPy)	0.4224	0.4018	0.3208	0
Gradient descent (2000 it.)	0.4224	0.4018	0.3208	3.7×10^{-15}
Heavy-ball (2000 it.)	—	—	—	4.0×10^{-15}
scikit-learn LinearRegression	—	0.4018	0.3208	4.2×10^{-15}

- All methods agree to machine precision ($\sim10^{-15}$); GD matches scikit-learn to 8.9×10^{-16}.
- This validates both the gradient formula and the convergence conditions.
- Baseline (predict the training mean): test MSE 0.6040.

- We compared our code with the exact formula and with scikit-learn's
- LinearRegression. All give the same weights to about 15 decimal places, so our
- implementation is correct. Test MSE 0.40 also beats the baseline of always guessing the average score (0.60).

13. Interpretation and limitations

- **Predictive quality is modest:** $R^2 \approx 0.32$, RMSE ≈ 0.634 vs label std ≈ 0.81. Causes: nonlinearity, ordinal and imbalanced labels (not Gaussian), subjective scores — limits of the model, not of GD.
- **Largest standardised coefficients:** alcohol $+0.289$, volatile acidity -0.198, sulphates $+0.148$, fixed acidity $+0.115$, total SO_2 -0.102. These are correlations, not causation.
- **Multicollinearity:** $\lambda_{min} = 0.0597$ keeps $\kappa \approx 50$ and makes single coefficients less stable.
- **Limits:** one train/test split; full-batch GD only.
- **Extensions:** cross-validation, mini-batch SGD, Nesterov, $\ell 2$ / Ridge (eigenvalues shift to $\lambda_i + \alpha$), polynomial features or ordinal classification.

- $R^2 = 0.32$ means the model explains about one third of the variation in scores: useful but limited, since taste is not purely linear and scores are subjective.
- More alcohol and sulphates go with higher scores, more volatile acidity with lower
- ones. This is correlation, not proof of cause.

14. Conclusions

- Gaussian likelihood ⇒ least squares ⇒ convex quadratic with a unique minimiser.
- GD converges **iff $\gamma < 2/\lambda_{max}$**; speed is governed by κ — experiments match theory (divergence at $2.05/\lambda_{max}$, bounds 401/803 vs 285/507 measured).
- **Standardisation is essential**: κ from 10^{10} to 51.6.
- **Momentum** gives $\approx 9\times$ speed-up ($\kappa \to \sqrt{\kappa}$).
- Our from-scratch GD equals the closed form and scikit-learn to $\sim 10^{-15}$.

- The mathematics predicted exactly what we observed: a safe step-size limit, speed controlled by κ, and big gains from rescaling and momentum.
- In the end, gradient descent finds the same answer as the closed-form formula.